2B: Carseats, categories, and class probabilities

Problem statement and worked resolution

Companion to `02_B_carseats.ipynb`

How to use this explanation. This explanation covers categorical preprocessing, tree selection, interpretable paths, and cost-sensitive decisions. The final probability model is allowed to be logistic regression if the training comparison favors it. Section numbers follow the notebook's numbered sections where present. Numerical values are rounded and refer to the recorded run, not universal performance claims.

Problem and notation

The local Carseats data contain 400 stores or markets. Define high sales by

$$Y = \mathbf{1}\{\text{Sales} > 8\},$$

where Sales is in thousands of units. There are 164 positive labels. The task is to predict high-sales probability from the remaining variables and then choose a class decision. Exclude `Sales` from every predictor set: otherwise the model would be given the quantity defining the answer. Use $p(x)$ for a model's positive-class probability, t for its threshold, and α for pruning strength. Cross-validation (CV) selects model settings and the probability model; separate validation data select a threshold. A Gini tree uses impurity $G(A) = 1 - \sum_k p_{Ak}^2$, where p_{Ak} is the class fraction in node A.

1 Load data and reserve partitions

`pd.read_csv` loads the course copy of `Carseats.csv`. Unlike Advertising, this CSV has no exported index column to discard. Run the notebook from the course root or `FocusedContent`; its local path lookup requires the bundled course data. The seven numerical features are CompPrice, Income, Advertising, Population, Price, Age, and Education. The three categorical features are ShelveLoc, Urban, and US. The code explicitly separates them and checks missing values and target counts.

Stratified splits with seed 23 give 240 training, 80 validation, and 80 test rows. Training contains 98 positives and 142 negatives, so its prevalence baseline is $98/240 \approx 0.4083$. Original row indices verify separation. Only training rows learn encodings, coefficients, or tree structure.

2 Build preprocessing inside each fold

Why encode categories?

A label such as shelf location Good is not a meaningful real number. Arbitrary codes like Bad=0, Medium=1, Good=2 would impose an order and spacing on threshold comparisons. One-hot encoding instead creates a zero-or-one column for each learned category. A test `ShelveLoc_Good <= 0.5` then means the row is not in the Good category among known labels.

Main Python calls

`OneHotEncoder(handle_unknown="ignore", sparse_output=False)` learns category columns and returns a dense array. An unseen category gives zeros for that feature's category indicators. `ColumnTransformer` applies different transformations to named column groups. `Pipeline` fits

preprocessing and the estimator together. `StandardScaler` scales numerical features for the logistic comparison; tree numerical features pass through unchanged. The same fitted transformation must process evaluation rows. For each CV fit, `fit` relearns the encoder and, for logistic regression, the scaler using that fold's fitting rows. Refitting the pipeline replaces previous fitted state; it does not incrementally retain earlier folds. This prevents preprocessing leakage.

Probability metrics

The selection criterion is Brier score,

$$\text{Brier} = \frac{1}{m}\sum_i (p_i - y_i)^2.$$

It remains finite at probabilities zero and one. Reported log loss is

$$-\frac{1}{m}\sum_i \{y_i \log \tilde{p}_i + (1 - y_i)\log(1 - \tilde{p}_i)\}, \qquad \tilde{p}_i = \min(1 - 10^{-12}, \max(10^{-12}, p_i)).$$

Clipping prevents infinite reported loss when a pure leaf is confidently wrong. It is a reporting convention applied equally to all candidates, not smoothing learned by the tree. A large clipped log loss can still reveal serious overconfidence.

3 Select tree settings and the probability model

`StratifiedKFold` supplies five splits of the 240 training rows: 192 fit and 48 score. Reuse the same folds for every candidate, calculating mean Brier score.

- Search `DecisionTreeClassifier(criterion="gini")` over depths 1, 2, 3, 5, unlimited, and minimum leaf sizes 2, 5, 10, 20.
- Separately search pruning alpha in $\{0, 0.001, 0.005, 0.01, 0.02, 0.05\}$ with minimum leaf size 2. Classification pruning uses weighted leaf Gini plus alpha times leaf count.
- Compare both tree winners with a logistic pipeline at fixed $C = 1$ and a fold-specific prevalence baseline.

C is inverse regularization for logistic regression. It is fixed here, so the logistic candidate is not an exhaustively tuned competitor. Exact ties use table order. Fold standard deviation is descriptive.

Candidate	Mean CV Brier
Prevalence baseline	0.241656
Logistic, $C = 1$	0.082353
Depth/leaf tree	0.186140
Pruned tree	0.222230

The depth/leaf winner has unrestricted maximum depth and minimum leaf size 10. The pruning winner has alpha 0.05. Logistic regression wins the probability comparison by a wide margin in this run. A lecture about trees does not require a tree to win. The chosen model family and settings are frozen, then each prespecified candidate is fitted on all 240 training rows. `probabilities(name, inputs)` applies a stored pipeline or returns the training prevalence. The repeated search uses CV for selection; it is not an independent assessment of the winning score, which is why test evaluation remains necessary.

4 Read a shallow tree and translate paths

A separate depth-two tree with minimum leaf size 10 is fitted for illustration. It does not replace the selected logistic model. `plot_tree` draws nodes, and `get_feature_names_out` attaches transformed feature names. `apply` identifies the terminal leaf reached by a row; `decision_path` returns the nodes it visited. The notebook translates two paths:

1. Advertising ≤ 7.50 and Price ≤ 92.00: estimated high-sales probability 0.733.
2. Advertising > 7.50 and ShelveLoc is Good: estimated probability 0.933.

Each path is a conjunction: every condition must hold. Leaf probability is the training positive fraction in that leaf, not a statement that each store's outcome is certain. Leaf counts are shown because probabilities based on few observations are less stable. These paths are predictive

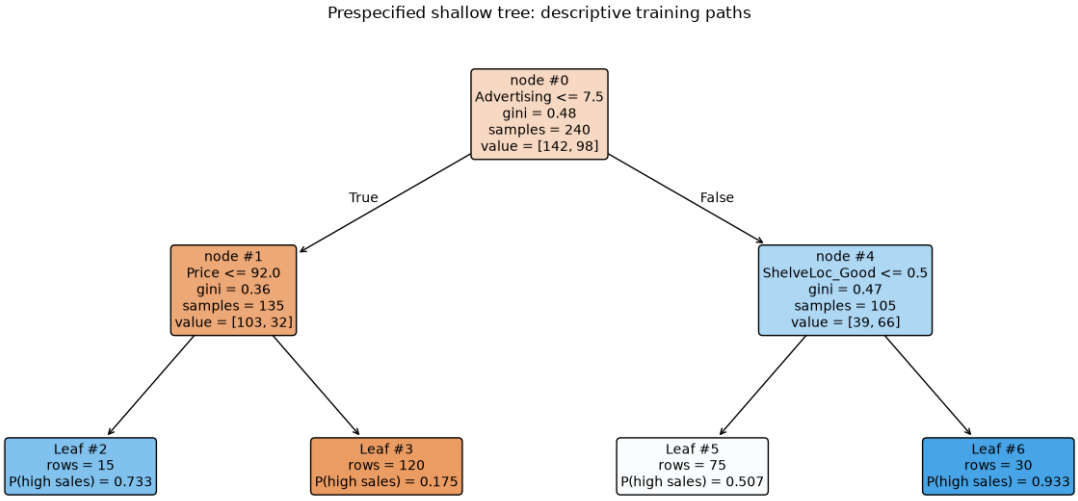


Figure 1: The descriptive shallow tree shows conditional rules and leaf frequencies. It is an interpretation example, not the CV-selected probability model.

associations in a teaching dataset. They do not imply that changing a store's advertising or shelf location would cause the predicted change in outcome.

5 Audit validation probabilities and choose a threshold

Logistic validation Brier is about 0.0799 and clipped log loss about 0.2741. The depth/leaf tree has Brier about 0.2076 and clipped log loss about 1.8238, illustrating how some confident mistakes can hurt log loss substantially.

For the selected model, `pd.cut` places probabilities into five fixed intervals from 0 to 1. Each plotted point compares a bin's mean prediction and observed positive fraction; empty bins are omitted and counts are shown. For example, the 0.6–0.8 bin contains only four stores, all positive. Its event rate 1 is a noisy observation, not proof that all predictions around 0.7 are too small.

The example specifies false-positive cost 1 and false-negative cost 3. If p is true risk, predicting positive costs $1 \times (1-p)$ in expectation, while predicting negative costs $3p$. The theoretical threshold is therefore $1/(1+3) = 0.25$, under zero cost for correct decisions. For each grid threshold 0.01 through 0.99, measure validation cost $FP + 3FN$. Dividing by 80 would not change the minimizing threshold. Equality predicts class 1; ties select the largest threshold. Validation selects $t = 0.31$,

demonstrating that an empirical optimum need not equal the theoretical cutoff. Keep the fitted probability model unchanged after this choice.

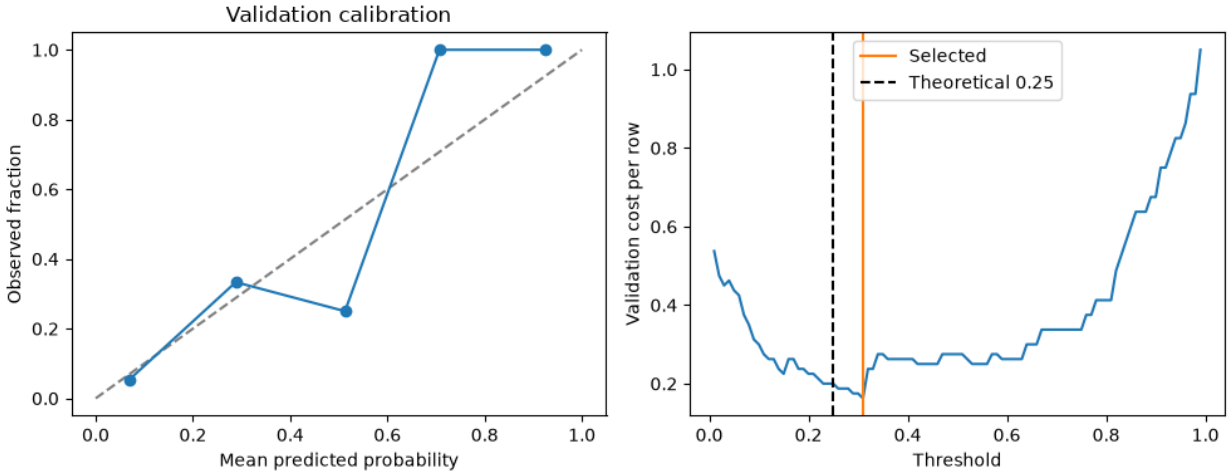


Figure 2: Validation calibration and measured cost. Both depend on only 80 rows; the chosen threshold is subject to sampling variation.

6 Final test probabilities and policies

Frozen candidate	Test Brier	Clipped log loss
Prevalence	0.242361	0.677791
Logistic	0.096407	0.307515
Depth/leaf tree	0.265255	3.240530
Pruned tree	0.272976	0.747249

The tree candidates perform worse than the prevalence baseline on this test probability metric. Report that result rather than selecting a different tree after seeing it. The selected logistic probabilities give:

Policy	TN	FP	FN	TP	Accuracy	Cost/row
Threshold 0.5	41	6	5	28	0.8625	0.2625
Threshold 0.25	34	13	2	31	0.8125	0.2375
Validation threshold 0.31	37	10	2	31	0.8500	0.2000
Prevalence at 0.25	0	47	0	33	0.4125	0.5875

The selected policy has precision $31/(31+10) \approx 0.7561$ and recall $31/(31+2) \approx 0.9394$. Its cost is $(10 + 3 \times 2)/80 = 0.20$. It improves cost relative to 0.5 despite slightly lower accuracy, because misses are weighted more heavily. The prevalence model predicts above 0.25 for everyone, hence raises an alarm for every row.

Answers to the interpretation questions

Logistic regression wins training CV. Small pure leaves can be overconfident, and one-hot rules describe category membership. The validation threshold beats 0.25 here but need not do so on another sample. These predictive associations do not establish causal effects.

Further explanation: categories, probabilities, and selection

Why numerical category codes change the available rules

Suppose shelf locations are coded Bad=0, Medium=1, Good=2. A threshold tree can directly separate Bad from the other two or Good from the other two, but cannot separate Medium from both others with one threshold. Changing the arbitrary codes changes which groups can be separated in one step. One-hot encoding lets each category be tested explicitly. It still does not search all possible multi-category groupings in a single split; several nodes may be needed for a union of categories.

If all category columns plus an intercept are supplied to an unpenalized linear model, there is exact redundancy because the indicators sum to one for known categories. The regularized logistic baseline can still be fitted, but individual category coefficients must be interpreted relative to the chosen representation and penalty. This is one reason the notebook focuses on predictions rather than treating that coefficient table as a unique set of category effects.

An unseen category is not a learned "other" category

With unknown categories ignored, an unseen shelf label produces zeros in all learned shelf columns. That lets the pipeline run, but the model has not learned a special response for the new label. A tree may route it down several "not this category" branches; a linear model gets no shelf contribution from those indicators. Neither behavior is a statistical guarantee for a category absent during training. The OneHotEncoder documentation is therefore relevant beyond merely finding the syntax: inspect the unknown-category behavior and the learned category list.

Why Brier and Gini are related but serve different roles

For a binary leaf with positive fraction r, using its mean as the probability gives

$$\frac{1}{n_A}\sum_{i\in A}(y_i - r)^2 = r(1-r), \qquad G(A) = 2r(1-r).$$

Thus a Gini split and a squared-probability-error split rank candidate binary partitions the same way on fitting data. That does not mean training Gini equals held-out Brier: validation rows were not used to determine leaf structure or proportions, and they may have different label frequencies.

A pure leaf predicts 0 or 1 and has zero training Gini. If it predicts 1 for a negative validation row, its Brier loss is 1 and its clipped log loss is about $-\log(10^{-12}) = 27.63$. One such row among 80 contributes about 0.345 to average clipped log loss. This concrete scale helps explain why the depth/leaf tree's log loss can be much worse than its Brier score might suggest.

Selecting a probability model and selecting a decision policy

A model is first selected by CV Brier, which asks for accurate probabilities across all rows. A threshold is then selected under $FP + 3FN$, which asks for useful decisions under specified costs. These are distinct objectives. Refitting the model after threshold selection could change its score distribution and invalidate the threshold fitted to the old predictions. The notebook consequently keeps the fitted pipeline unchanged.

The best tree setting has already won a search over many possibilities. Comparing that winning CV score with another model is useful for selection, but treating it as an unbiased external performance estimate ignores the search. The final 80-row test is the reserved assessment; its small size also explains why modest differences are uncertain.

Practical tips for using pipelines and plots

Keep the encoder attached to its fitted estimator. Encoding test data independently can change column order, column count, or category meanings even when the arrays appear numerically plausible. The scikit-learn pitfalls guide explains this train/evaluation separation and the role of pipelines. A tree plot's node values and class order should be checked against the model rather than assumed from colors. Using node sample counts and explicit positive fractions makes the diagram easier to interpret across software versions. For real deployment, additionally ask whether the prediction time precedes all recorded inputs and whether new stores or later periods differ from the randomly held-out records studied here.